Improvement Plan: Guardrail Agentic Research Toolkit

Extensibility, Reproducibility, and Production Readiness

April 2026

Overview

This document proposes improvements to the **guardrail_agentic** research toolkit, which compares *agentic* (with web retrieval tools) vs *non-agentic* (single LLM call) guardrail safety evaluations on asylum/humanitarian scenarios in English and Farsi.

The toolkit works well for its current scope (60 scenarios, 3 tool types, 3 guardrail backends), but has architectural limitations that make it difficult to extend with new tools, experiment configurations, or to scale toward production use. The improvements below are organized into five workstreams that collectively make the package **extendable** and **productionalizable**.

Key Issues Identified

Category	Issue	Impact
Architecture	No abstractions (string-based tool dispatch, class-name backend matching)	Adding a tool requires changes in 3 files
Architecture	Scattered configuration (magic numbers in 4+ files, CLI args, .env)	Hard to reproduce or audit experiments
Architecture	Empty `__init__.py`; not usable as a library	Cannot import programmatically
Architecture	Hardcoded system prompts in Python code	Cannot test prompt variants without code changes
Experimental	Single comparison axis (agentic vs non-agentic only)	No ablation studies possible
Experimental	No experiment config files	Cannot save/replay experiment settings
Experimental	No statistical analysis	Per-scenario deltas only; no significance tests
Reliability	Zero automated tests, no CI	Refactoring carries high regression risk

Category	Issue	Impact
Reliability	No checkpointing or error recovery	Mid-run crash loses all progress
Performance	Fully sequential row processing	Unnecessarily slow for 60+ scenarios
Performance	No caching or rate limiting	Repeated queries; DuckDuckGo rate-limits
Observability	Logging hardcoded to filesystem	Cannot plug in MLflow, W&B, or databases

Workstream 1: Core Abstractions & Configuration

Priority: HIGH — This is prerequisite infrastructure for all other workstreams.

1a. Centralized Configuration

New file: `agentic_guardrails/config.py`

Create an `ExperimentConfig` dataclass (or Pydantic model) that collects every parameter currently scattered across module-level constants, CLI arguments, and hardcoded values:

- `max_tool_calls` — currently `agentic_runner.py` line 33, default 5
- `valid_score_threshold` — currently `agentic_runner.py` line 34, default 0.5
- `max_fetch_chars` / `max_search_results` — currently `tools.py` lines 19–20
- `http_timeout` — hardcoded as `timeout=10` in `tools.py`
- Provider/model settings, policy paths, rubric path, log directory, output prefix

The config object should support YAML/JSON serialization (`save(path)` / `load(path)`) for experiment reproducibility. Add a `--config-file` CLI flag as an alternative to individual arguments.

1b. Tool Registry

Refactor: `agentic_guardrails/tools.py`

Define a `Tool` Protocol:

```
class Tool(Protocol):
    name: str
    schema: dict   # OpenAI-format function schema
    def execute(self, **kwargs) -> str: ...
```

Create a `ToolRegistry` class with `register()`, `dispatch()`, and `get_schemas()` methods. Convert the three existing tools (`search_web`, `fetch_url`, `check_url_validity`) into classes implementing this protocol.

Result: Adding a new tool requires writing one class and calling `registry.register(MyTool())`. No changes needed in `agentic_runner.py` or the dispatch function.

1c. Guardrail Backend Abstraction

Refactor: `agentic_guardrails/guardrails_runner.py`

Replace the class-name-based backend detection (line ~183: `backend_name = guardrail.__class__.__name__.lo` with a `GuardrailAdapter` Protocol providing a uniform `evaluate(eval_text, policy_text)` interface. Create `FlowJudgeAdapter`, `GliderAdapter`, and `AnyLLMAdapter` classes, each wrapping its backend's distinct `.validate()` signature.

1d. System Prompt Externalization

Refactor: `agentic_guardrails/agentic_runner.py`

Move `build_agentic_guardrail_system_prompt()` (lines 63–106), `_CONCLUDE_MESSAGE`, and `_RETRY_MESSAGE` out of Python code and into Jinja2 templates under `config/templates/`.

Templates receive variables like `policy`, `rubric`, and `tool_descriptions` (auto-generated from the tool registry). This enables testing prompt variants without code changes.

1e. Package API

Modify: `agentic_guardrails/__init__.py`

Export the public API: `ExperimentConfig`, `run_agentic_guardrail`, `AgenticJudgment`, `ToolRegistry`, `compare_judgments`, `ComparisonResult`. This makes the package importable as a library, not just runnable as a CLI script.

Workstream 2: Testing & CI

Priority: HIGH — Safety net for all refactoring. Currently zero tests exist.

2a. Project Packaging

New file: `pyproject.toml`

Consolidate `requirements.txt` and `agentic_guardrails/requirements_agentic.txt` into a single package definition with optional dependency groups (`test`, `dev`, `viz`). Configure pytest settings.

2b. Unit Tests

New directory: `tests/`

Target the highest-value pure functions first:

- **`test_comparison.py`**: All edge cases for `compare_judgments` — None scores, delta signs, judgment flips
- **`test_parse_judgment.py`**: Fenced JSON, bare JSON, nested braces, missing fields, unparseable text
- **`test_output_writer.py`**: Round-trip write and read-back
- **`test_tools_registry.py`**: Registration, schema generation, dispatch (after 1b)
- **`test_config.py`**: Config serialization/deserialization (after 1a)

2c. Integration Tests

New file: `tests/test_integration.py`

Mock `any_llm.completion` and network calls, then run `process_row` end-to-end. Verify that tool calls are correctly capped, errors in one path don't crash the other, and token accumulators work across turns.

2d. CI Pipeline

New file: `.github/workflows/ci.yml`

Run `pytest` and `ruff check` on every push and pull request.

Workstream 3: Experiment Design & Reproducibility

Priority: MEDIUM-HIGH — Directly serves the research use case.

Depends on: Workstream 1 (config object, tool registry for subsetting)

3a. Experiment Configuration Files

New file: `agentic_guardrails/experiment.py`

Extend `ExperimentConfig` with:

- `experiment_id` — auto-generated UUID + timestamp for tracking
- `description` — human-readable notes
- `seed` — for future stochastic components
- `ablation` sub-config: `enabled_tools` (subset of registered tools), `system_prompt_variant` (path to alternative template), `rubric_variant`

Support a `--save-config` / `--config-file` workflow:

```
# Save a config for later replay
python run_agentic_comparison.py --save-config experiments/no_search.yaml ...

# Replay it exactly
python run_agentic_comparison.py --config-file experiments/no_search.yaml
```

3b. Checkpointing & Resume

Modify: `run_agentic_comparison.py`

Write a checkpoint file (`<output_prefix>_checkpoint.json`) after each completed row. Add a `--resume` flag that skips already-completed rows. This prevents data loss when a run crashes mid-way (currently the entire run must be restarted from scratch).

3c. Statistical Analysis Module

New file: `agentic_guardrails/analysis.py`

Replace per-scenario-only deltas with aggregate statistics:

- Mean, median, standard deviation of `score_delta` per policy
- Paired Wilcoxon signed-rank test for significance
- Bootstrap confidence intervals for mean delta
- Breakdown by language (EN vs FA), tool usage, and judgment change
- Cohen's d effect size

Uses existing `scipy` and `numpy` dependencies. Callable from both the CLI (`--analyze` flag) and the Streamlit dashboard.

3d. Ablation Study Runner

New file: `agentic_guardrails/ablation.py`

Orchestrator that takes a list of `ExperimentConfig` variants and runs each one, collecting results into a comparison table. Example ablation axes:

- **Tools**: all three vs `search_web` only vs none
- **Prompts**: default vs minimal vs detailed
- **Models**: gpt-5-mini vs gpt-5 vs claude-sonnet
- **Rubrics**: 0–1 scale vs 1–5 FlowJudge scale

Results saved in `experiments/<experiment_id>/` alongside the config YAML.

Workstream 4: Performance & Reliability

Priority: MEDIUM — Current 60-scenario dataset is manageable, but ablation grids require parallelism.

Depends on: Workstream 1 (config object for new parameters)

4a. Parallel Execution

Modify: `run_agentic_comparison.py`

Add `concurrent.futures.ThreadPoolExecutor` with a `--workers N` flag (default 1 for backward compatibility). Each row is independent and the bottleneck is I/O (LLM API calls, web requests), making threading ideal with minimal refactoring.

4b. Rate Limiting

New file: `agentic_guardrails/rate_limiter.py`

Token-bucket rate limiter for LLM API calls (per-provider RPM/TPM limits) and DuckDuckGo searches (currently has no rate limiting and can return 429 errors). Configurable in `ExperimentConfig` with sensible defaults.

4c. Response Caching

New file: `agentic_guardrails/cache.py`

Optional disk-based cache (SQLite) for web search results, URL validity checks, and assistant LLM responses. Key = query/URL/hash, value = result + timestamp, with configurable TTL. Enabled with `--cache-dir <path>`. Avoids repeating identical DuckDuckGo queries across scenarios and enables cheap re-runs when only the guardrail prompt changes.

4d. Retry with Backoff

Modify: `tools.py` and `providers.py`

Replace the hardcoded single retry in `agentic_runner.py` (lines 387–428) with configurable exponential backoff. Apply to LLM API calls (transient 429/500/503 errors), web fetches (network timeouts), and DuckDuckGo searches (rate limiting). Retry count and backoff parameters configurable in `ExperimentConfig`.

Workstream 5: Observability & Dashboard

Priority: MEDIUM-LOW — Current filesystem logging works for individual runs; becomes important at scale or with team collaboration.

Depends on: Workstream 1 (config), Workstream 3c (analysis module)

5a. Logger Abstraction

Refactor: `agentic_guardrails/scenario_logger.py`

Define a `LoggerBackend` Protocol with the same method signatures as `ScenarioLogger`. The current implementation becomes `FileLoggerBackend`. Add:

- **`JsonLinesLoggerBackend`**: One JSONL line per event (streaming, crash-safe)
- **`CompositeLoggerBackend`**: Fans out to multiple backends simultaneously
- Future: MLflow and Weights & Biases adapters

Backend selected via `ExperimentConfig.log_backend` (default: `"file"`).

5b. Run Metrics Summary

Modify: `run_agentic_comparison.py`

After the pipeline completes, emit `<output_prefix>_metrics.json` containing: total scenarios, success/error counts, mean/median score deltas per policy, total tokens consumed, total tool calls, and wall-clock runtime.

5c. Dashboard Improvements

Modify: `visualize_results.py`

Add a statistical analysis tab (from 3c) with paired scatter plots, delta histograms, and inline significance test results. Add an ablation comparison view that loads multiple experiment outputs for side-by-side comparison.

Note on Agentic Framework Adoption

The hand-rolled tool-calling loop in `agentic_runner.py` is only approximately 120 lines of core logic and is tightly coupled to research-critical behaviors: the two-phase verification protocol, conclude/retry messages, and per-turn token tracking. Adopting an external framework (LangGraph, Claude Agent SDK, etc.) would:

- Add heavyweight dependencies with lock-in risk
- Require significant customization to preserve all research behaviors
- Not solve the actual pain point (adding tools), which the Tool Registry (1b) addresses directly

Recommendation: Do not adopt a framework now. The abstractions in Workstream 1 will make migration straightforward later if the need arises.

Implementation Sequence

Phase	Workstreams	Focus
Phase 1	1a–1d + 2a–2b	Config, abstractions, packaging, unit tests
Phase 2	1e + 2c–2d + 3a–3b	API exports, integration tests, CI, experiment configs
Phase 3	3c–3d + 4a–4b	Analysis, ablation, parallelism, rate limiting
Phase 4	4c–4d + 5a–5c	Caching, retry, observability, dashboard

Critical Files

File	Workstreams
`agentic_guardrails/agentic_runner.py`	1b, 1d, 4d
`agentic_guardrails/tools.py`	1b, 4b–4d
`agentic_guardrails/run_agentic_comparison.py`	1a, 3a–3b, 4a, 5b
`agentic_guardrails/guardrails_runner.py`	1c
`agentic_guardrails/scenario_logger.py`	5a
`agentic_guardrails/comparison.py`	3c
`visualize_results.py`	5c

Verification Plan

- **Unit tests**: `pytest tests/` passes for all pure functions
- **Integration test**: Mock LLM + mock network leads to full `process_row` end-to-end success
- **Config round-trip**: Save config YAML, load, run — identical results
- **Checkpointing**: Kill mid-run, then `--resume` completes remaining rows
- **Parallelism**: `--workers 4` produces same results as `--workers 1`

- **Tool registry**: Register a dummy tool; it appears in schemas and dispatches correctly
- **Ablation**: Run with `enabled_tools: [search_web]` only; `check_url_validity` is never called